

A Small, Typed, Staged Way to Solve Problems

A reusable prompt template derived from the pyramid database session. It combines a domain-specific language, an explicit semantic specification, type-indexed validation, and Futamura-style partial evaluation.

Core idea

Separate what is known now from what must remain dynamic. Parse and validate the specification while generating the artifact; specialize away everything fixed; leave only residual runtime behavior in the final program

```
Surface input -> typed semantic specification -> reference interpreter + specializer -> residual artifact
```

General prompt template

You are designing a small, auditable implementation for this problem:

[PROBLEM]

1. Define the smallest surface DSL users need. State its grammar and valid/invalid examples.
2. Define one declarative schema. Each named value has a semantic kind, representation, and invariant.
3. Define the semantic domain mathematically or with a compact algebraic data type.
4. Separate stages: parse : Surface -> Parsed; validate : Parsed -> Validated or Error; interpret : Validated x Input -> Result; specialize : Validated -> ResidualProgram.
5. Use type indices, phantom types, GADTs, dependent types, or checked constructors only where they prevent a real invalid state. Erase them after validation.
6. Apply the first Futamura projection: partially evaluate the interpreter with respect to the validated static specification. List what disappears from the output.
7. Keep HTTP, databases, and files behind adapters. Keep the semantic core deterministic and dependency-light.
8. Generate the smallest readable target program: only dynamic data, residual control flow, and runtime effects.
9. Verify by differential testing: reference interpreter result == residual program result. Test every type rule with invalid input.
10. Report uncertainty, unsupported grammar, missing data, and empirical versus formal claims.

Deliver the grammar, semantic specification, staging boundaries, interpreter, specializer, generated artifact, and verification commands.

Review checklist

Question	Pass condition
Static?	Schema, validated query, limits, predicates, and output shape are fixed before specialization.
Dynamic?	Only genuinely unknown records, values, or effects remain.
Error boundary?	Invalid syntax and types are rejected before specialization.
Residual?	The output has no unnecessary parser, AST, or dynamic dispatch.
Evidence?	Claims are supported by types, executable checks, or explicit finite evidence.
External?	Network and acquisition logic are visible adapters.

Session-derived rules

1. Use one schema as the semantic source of truth.
2. Prefer a first-order DSL over a general SQL engine when the problem needs only selection, predicates, and a limit.
3. Represent type-sensitive cases explicitly, such as TextPredicate and NumberPredicate.
4. Erase semantic types only after validation, when emitting residual target code.
5. Keep the generated program free of the generator's parser and interpreter.
6. Do not add a third Futamura projection unless it genuinely makes the system smaller and clearer.

Worked abstract shape

For a query-driven data tool the compact semantic normal form is:

```
Query = (Predicate*, Limit)
eval(Query, data) = take(limit, filter(data, conjunction(predicates)))
```

The specialized receives only a validated query and emits a residual loop. This is the practical first Futamura projection:

```
generic evaluator + fixed validated query -> residual program
```

The method generalizes to parsers, validators, search tools, data transforms, protocol compilers, and configuration-driven programs. The central discipline is to make semantic boundaries explicit and avoid carrying static generality into the runtime artifact.